

Python Twitter 情緒分析 報告擴展

資料整理、情緒評分
與視覺化流程

製作人 B12204033 施邵

期末專題簡報

報告大綱

- 1 專題介紹
- 2 資料與系統設計
- 3 核心程式碼講解
- 4 分析成果
- 5 結論

研究背景與動機

- 社群媒體在公共議題傳播中具有即時、密集且可量化的特性。
- Twitter 貼文可作為觀察民眾情緒變化的重要資料來源。
- 本專題希望以 Python 建立一套完整流程，從資料清理、情緒分析到視覺化輸出。
- 目標不只在於得到結果，也要讓分析程式具備可重跑、可維護與可展示的特性。

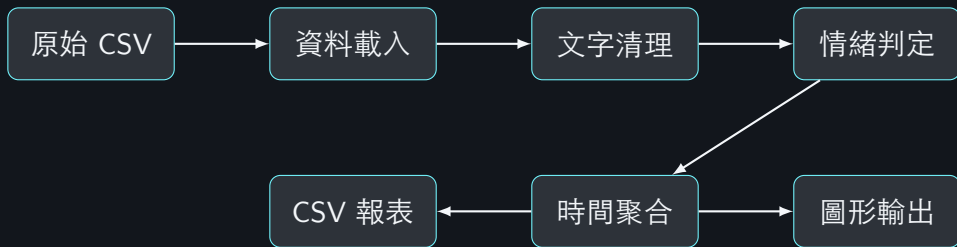
專題目標與研究問題

專題目標

建立一個可重複執行的 Twitter 情緒分析流程，並輸出適合期末報告展示的圖表與摘要報表。

- 如何清理大量推文文字，保留可分析的語意資訊？
- 如何以 VADER 模型為基礎，將推文分類為正向、中立、負向？
- 如何觀察情緒比例在不同時間區段的變化？
- 如何讓最終成果兼具分析價值與簡報可讀性？

專題流程總覽



- 核心流程集中於 `twitter_sentiment_project/` 套件。
- 最終輸出包含分析圖、關鍵字圖、主題分布圖與多份 CSV 摘要。

資料來源與資料特徵

資料概況

- 原始資料筆數：82,309
- 有效分析筆數：82,035
- 無效或重複資料移除：274
- 分析日期：2021-01-06
- 主要欄位：`tweet_id`、`query`、`text`、`created_at`

資料特性

- 屬於混合主題資料集
- 每個 `query` 的筆數相近
- 適合用於比較整體情緒與時間變化
- 需先完成欄位驗證與時間格式轉換

專案架構重整

主要資料夾

<code>data/raw</code>	原始資料
<code>data/processed</code>	清理後資料
<code>outputs/figures</code>	圖形成果
<code>outputs/reports</code>	分析報表
<code>slides</code>	簡報檔案

核心模組

<code>data_loader.py</code>	載入資料
<code>preprocessor.py</code>	清理文字
<code>sentiment_analyzer.py</code>	情緒評分
<code>analyzer.py</code>	聚合與摘要
<code>visualizer.py</code>	圖表繪製
<code>pipeline.py</code>	串接流程

執行環境與套件選擇

主要技術

- Python 3
- pandas
- matplotlib
- vaderSentiment
- LaTeX Beamer

設計理由

- pandas 處理大量表格資料穩定
- VADER 適合短文字情緒分析
- matplotlib 可輸出高解析度 PNG
- Beamer 適合正式學術簡報

程式碼一：資料載入與欄位驗證

```
1 def load_data(file_path=None):
2     path = Path(file_path) if file_path else resolve_default_input_file()
3     df = pd.read_csv(path)
4     prepared = df.copy()
5     prepared["text"] = prepared["text"].astype(str)
6     prepared["created_at"] = pd.to_datetime(
7         prepared["created_at"], errors="coerce", utc=True
8     )
9     prepared = prepared.dropna(subset=["created_at", "text"]).copy()
10    prepared = prepared.drop_duplicates(subset=["tweet_id"])
11    return prepared.sort_values("created_at").reset_index(drop=True)
```

- 先確認資料路徑，再讀取 CSV。
- 將 created_at 統一轉換為可排序的時間格式。
- 刪除空值與重複資料，確保後續統計一致。

程式碼二：文字清理與斷詞

```
1 def clean_tweet(text):
2     normalized = html.unescape(str(text)).replace("\n", " ")
3     normalized = URL_PATTERN.sub(" ", normalized)
4     normalized = MENTION_PATTERN.sub(" ", normalized)
5     normalized = normalized.replace("#", " ").lower()
6     normalized = NON_TEXT_PATTERN.sub(" ", normalized)
7     return MULTISPACE_PATTERN.sub(" ", normalized).strip()
8
9 def apply_cleaning(df):
10     cleaned = df.copy()
11     cleaned["clean_text"] = cleaned["text"].apply(clean_tweet)
12     cleaned["tokens"] = cleaned["clean_text"].apply(tokenize)
13     cleaned["token_count"] = cleaned["tokens"].apply(len)
14     return cleaned
```

- 移除網址、帳號提及、特殊符號與多餘空白。
- 將推文轉為小寫，提升情緒分析與關鍵詞統計的一致性。
- 額外保留 `tokens` 與 `token_count` 供後續分析使用。

程式碼三：情緒分數與標籤判定

```
1 def get_sentiment_score(text):
2     analyzer = get_sentiment_analyzer()
3     return analyzer.polarity_scores(text)["compound"]
4
5 def label_sentiment(score, positive_threshold=0.05, negative_threshold=-0.05):
6     if score >= positive_threshold:
7         return "Positive"
8     if score <= negative_threshold:
9         return "Negative"
10    return "Neutral"
11
12 def apply_sentiment(df):
13     analyzed = df.copy()
14     analyzed["sentiment_score"] = analyzed["clean_text"].apply(get_sentiment_score)
15     analyzed["sentiment"] = analyzed["sentiment_score"].apply(label_sentiment)
16     return analyzed
```

- VADER 適合用來評估短文字情緒分數。
- 本專題以固定門檻區分正向、中立與負向。
- 快取機制可避免重複初始化分析器。

程式碼四：時間區段聚合

```
1 def compute_time_sentiment(df, freq="15min", min_tweets=10):
2     working = df.copy()
3     working["time_bin"] = working["created_at"].dt.floor(freq)
4     sentiment_counts = (
5         working.groupby(["time_bin", "sentiment"])
6         .size().unstack(fill_value=0)
7         .reindex(columns=SENTIMENT_LABELS, fill_value=0)
8     )
9     sentiment_counts["tweet_count"] = sentiment_counts.sum(axis=1)
10    for sentiment in SENTIMENT_LABELS:
11        sentiment_counts[f"{sentiment}_ratio"] = (
12            sentiment_counts[sentiment] / sentiment_counts["tweet_count"]
13        )
14    return sentiment_counts[sentiment_counts["tweet_count"] >= min_tweets]
```

- 以 15 分鐘為單位觀察情緒變化。
- 每個時間區段都會計算三種情緒的比例。
- 若推文數量過少，該區段會被排除以避免噪音影響。

程式碼五：管線整合與圖表輸出

```
1 def run_pipeline(input_file=None, output_dir=OUTPUT_DIR, freq="15min"):
2     raw_df = load_data(input_file)
3     cleaned_df = apply_cleaning(raw_df)
4     scored_df = apply_sentiment(cleaned_df)
5     time_df = compute_time_sentiment(scored_df, freq=freq, min_tweets=10)
6     distribution_df = compute_sentiment_distribution(scored_df)
7     top_terms_df = compute_top_terms(scored_df, top_n=12)
8     plot_sentiment_dashboard(time_df, distribution_df, summary_df, ...)
9     plot_top_terms(top_terms_df, ...)
10    plot_query_distribution(query_distribution_df, ...)
11    return PipelineOutputs(...)
```

- pipeline.py 將所有分析步驟整合為一個可重跑的流程。
- 除了圖片之外，也同步輸出多份 CSV 報表。
- 這種設計能兼顧程式維護、報告撰寫與成果展示。

程式碼六：主程式入口與執行方式

```
1 def main():
2     args = parse_args()
3     outputs = run_pipeline(
4         input_file=args.input,
5         output_dir=args.output_dir,
6         freq=args.freq,
7         min_tweets=args.min_tweets,
8         top_n_terms=args.top_n_terms,
9     )
10    print(f"Dashboard figure: {outputs.dashboard_path}")
11
12 if __name__ == "__main__":
13     main()
```

執行指令

```
python3 main.py
python3 main.py --freq 15min
python3 main.py --top-n-terms 12
```

意義

- CLI 參數化提高彈性。
- 可快速調整時間粒度與輸出設定。
- 便於之後擴充為不同資料集版本。

輸出成果與檔案

圖形輸出

- `sentiment_dashboard.png`
- `top_terms_by_sentiment.png`
- `query_distribution.png`

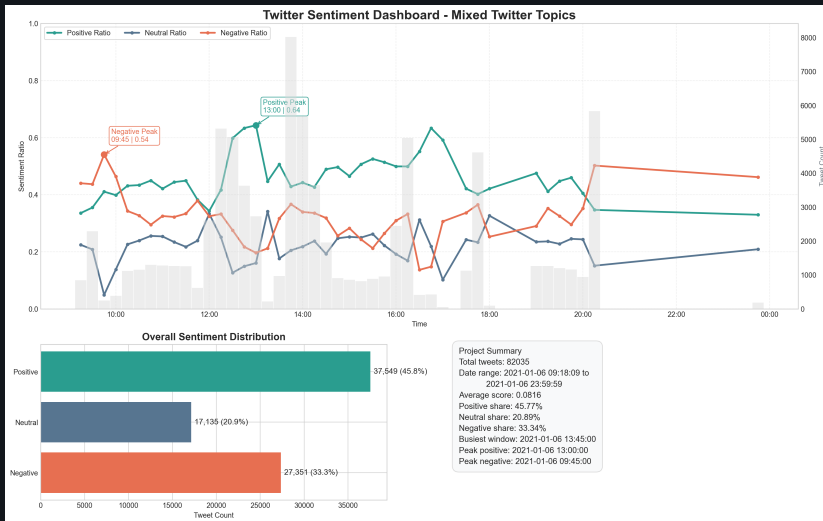
報表輸出

- `overall_summary.csv`
- `sentiment_distribution.csv`
- `time_sentiment_summary.csv`
- `top_terms_by_sentiment.csv`

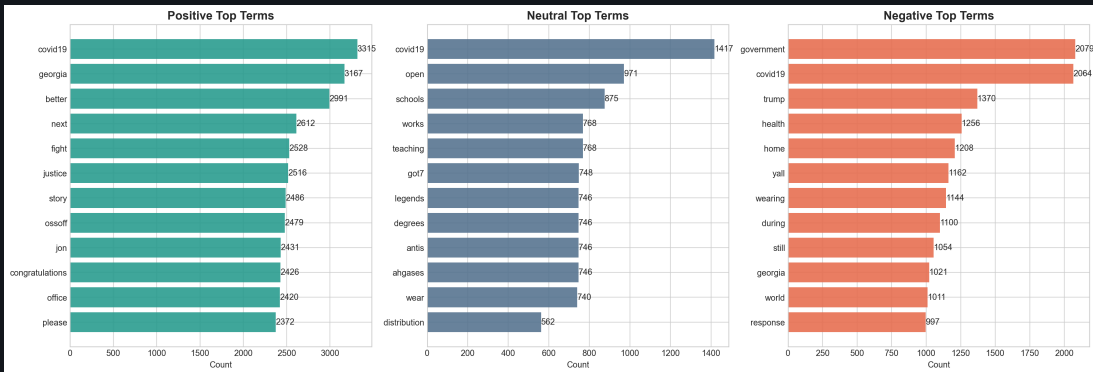
成果整理重點

本專題的成果不只是一張圖，而是一組可重複產生的分析資產，方便用於書面報告、簡報與後續延伸研究。

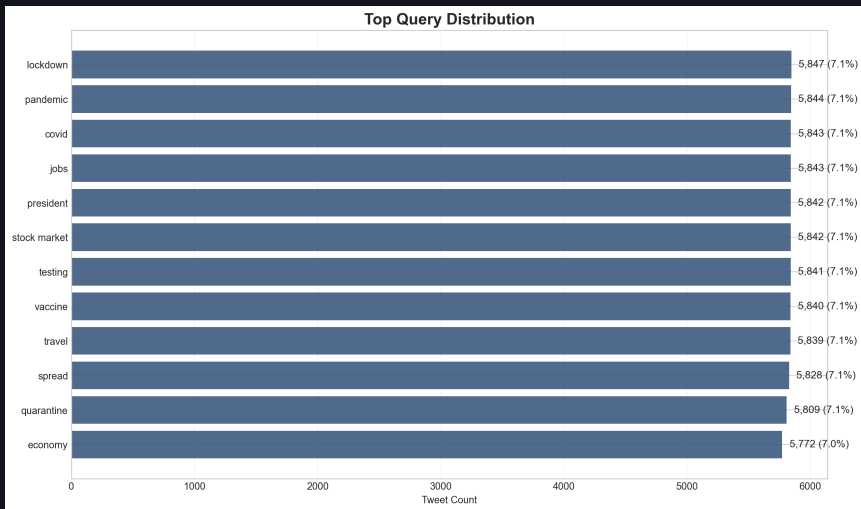
情緒儀表板結果



關鍵詞分析結果



主題分布結果



量化摘要與結果解讀

指標	數值
有效推文數	82,035
正向比例	45.77%
中立比例	20.89%
負向比例	33.34%
平均情緒分數	0.0816
最繁忙時段	13:45
正向高峰時段	13:00
負向高峰時段	09:45

結果解讀

- 整體正向比例高於負向比例。
- 不同時段的情緒波動相當明顯。
- 資料集屬於多主題混合，因此更適合觀察整體公共討論氛圍。

專題限制與後續改進

目前限制

- VADER 屬於字典式方法，對諷刺與脈絡理解有限。
- 本次資料只涵蓋單日推文，時間跨度較短。
- 中英文混雜或特殊語句可能影響判定精度。

後續方向

- 導入機器學習或深度學習情緒模型。
- 納入多日資料進行事件前後比較。
- 加入地區、主題與使用者層級的細部分析。

結論

- 本專題完成了從資料載入、清理、情緒判定到視覺化輸出的完整分析流程。
- 程式架構經過模組化整理後，更適合維護與延伸。
- 最終成果以圖表與報表雙軌輸出，提升了分析可讀性與簡報展示效果。
- 這份專題可作為 Python 資料分析與文字探勘應用的完整案例。

Q&A

感謝聆聽，敬請指教。